RL-Based Smart Parking Allocation

AI Assignment 2 — Real-Life Use Cases of Reinforcement Learning

Team: Duggirala Manoj (B24CM1022, b24cm1022@iitj.ac.in) and Likith Reddy (B24CM1069, b24cm1069@iitj.ac.in)
GitHub: (add repository link)

Abstract

We model smart parking allocation as an RL problem with three zones, uncertain vehicle stays, and changing occupancy. A Q-learning agent observes zone occupancy, vehicle stay category, and demand period, then selects a zone. Rewards balance search time, walking distance, and congestion. We compare its performance with simple allocation baselines.

1. Problem and objective

Parking allocation is difficult when vehicles arrive continuously, spaces have different walking distances, and parked vehicles leave at uncertain times. A fixed rule can overload one zone while others stay free. We model a small three-zone car park and train a tabular Q-learning agent to assign each arriving vehicle to a zone, aiming to reduce simulated search effort and congestion while keeping walking distance reasonable.

2. Environment

The custom Python simulator has three zones with capacities 5, 6 and 7 and walking-distance costs 1.8, 1.0 and 3.2 units. One vehicle arrives per time step with a short, medium or long stay (2–5, 6–12 or 13–24 steps). The stay mix changes across the day, and parked vehicles' remaining times decrease each step. An episode has 120 vehicle decisions.

3. RL formulation

State: three quantized zone-occupancy levels (0–3 each), vehicle stay category (0–2) and time segment (0–2): at most $4^3 \times 3 \times 3 = 576$ states. **Action:** choose one of the three zones; full zones are masked, and the environment penalizes an invalid choice. **Transition:** the vehicle's stay is added to the chosen zone, time advances, and all remaining stays drop by one.

4. Reward function

For a valid assignment: **r = 5** − **0.8S** − **0.45W** − **0.9C** − **D**, where S is simulated search time (rising with pre-assignment occupancy), W the zone walking distance, C a congestion penalty above 65% occupancy and D a small expected-stay blocking cost. An invalid action receives −12. The weights are simulator design choices, not measured human preferences.

5. Learning algorithm

Tabular Q-learning with epsilon-greedy exploration: $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max Q(s',a') - Q(s,a)]$, with no future term at terminal steps. We use $\alpha = 0.15$, $\gamma = 0.93$, epsilon decaying by 0.992 per episode to a minimum of 0.04, 1,800 training episodes and seed 7.

6. Baselines and evaluation

The trained policy is evaluated over 150 seeded episodes (seeds from 2000) against three baselines: first-available, nearest-available and least-occupied. All policies see the same episode seeds. Metrics: average search time, walking distance and congestion penalty per accepted vehicle, acceptance rate and invalid-action rate.

7. Results

Q-learning reached an average search cost of 1.80, versus 3.13 (first-available), 3.24 (nearest-available) and 1.66 (least-occupied). Mean walking distance was 1.91, 1.54, 1.34 and 2.06. Every policy accepted all arrivals, so acceptance rate does not separate them.

Policy	Search	Walk	Congest.	Accept
Q-learning	1.80	1.91	0.009	1.00
First available	3.13	1.54	0.290	1.00
Nearest available	3.24	1.34	0.327	1.00
Least occupied	1.66	2.06	0.002	1.00

Table 1. Mean metrics over 150 evaluation episodes (search and walk in simulator units).



Figure 1. Q-learning reward during training (60-episode moving average).

8. Policy comparison

The least-occupied baseline has slightly lower search cost than Q-learning in this run, while Q-learning cuts search cost by roughly 42% against first-available and 44% against nearest-available. The nearest- and first-available rules have the shortest walks but the highest congestion, whereas Q-learning and least-occupied spread load more evenly at the price of longer walks. The RL agent therefore does not beat every baseline on every metric; this is a small simulation study.

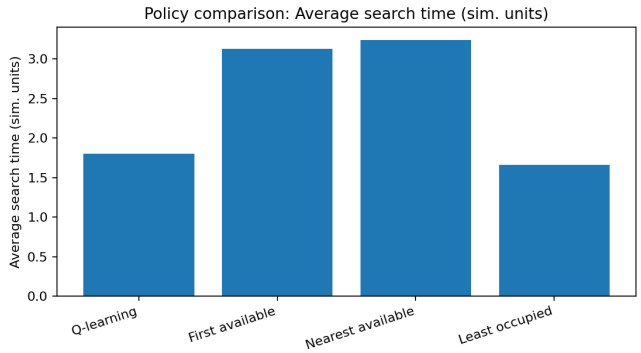


Figure 2. Average simulated search cost across policies.

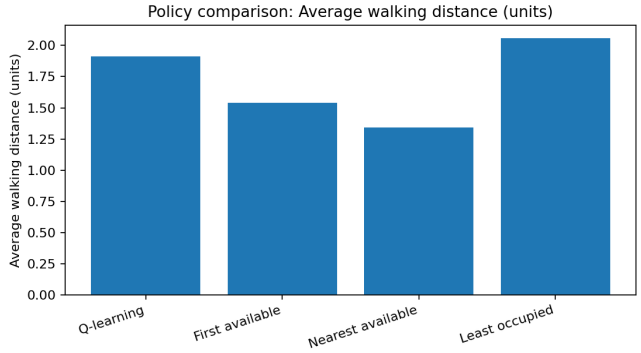


Figure 3. Average walking distance across policies.

9. Limitations

The environment is synthetic: stylized arrivals, stays, capacities and hand-designed reward weights. It uses no real parking data, models no individual routes and has no real-time sensors. No policy rejected a vehicle, so demand pressure should be raised in future tests. Results come from a single training seed.

10. Conclusion and next steps

We built a reproducible RL formulation for parking-zone assignment with a custom simulator, tabular Q-learning, three baselines, saved Q-table, CSV records and plots. The results show a trade-off between search effort and walking distance but do not establish superiority over rule-based methods. Future work: sensitivity tests over capacities and arrival rates, multiple training seeds, and calibration with permitted real parking data.

Reproduction

Install Python 3.10+ and **requirements.txt**, then run **python run_experiments.py** to train, evaluate, save the Q-table, write CSVs and regenerate figures (training seed 7, evaluation seeds from 2000). Contents: parking_env.py, rl_agent.py, run_experiments.py, trained_q_table.pkl, results/*.csv, figures/*.png, README.md and this report.

References

1. Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press.

2. Watkins, C. J. C. H., & Dayan, P. (1992). Q-learning. *Machine Learning*, 8, 279–292. https://doi.org/10.1007/BF00992698

3. The simulator, reward function and experiment code were implemented for this project; all reported values are generated by run_experiments.py with the documented seeds.

4. No external RL repository is used as the implementation base; the cited sources are conceptual references for the method.